

TypeScript – Complete Guide (30 LPA MERN Bootcamp)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 7 Sep 2026 – 13 Sep 2026 (Days 22–28)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm**, **3:00–5:30pm**, **6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import <calendar/mern-mastery-study.ics>

Target level: Senior frontend / full-stack interviews at product companies (Adobe, Salesforce, Razorpay, Flipkart, Google-adjacent roles). TypeScript is non-negotiable at 30 LPA — you must explain types, generics, and trade-offs fluently.

Table of Contents

1. [Why TypeScript at 30 LPA](#)
2. [Setup & Toolchain](#)
3. [All Core Types](#)
4. [Interfaces vs Types](#)
5. [Functions & Overloads](#)
6. [Generics — Deep Dive](#)
7. [Utility Types](#)
8. [Enums & Literal Types](#)
9. [Type Guards & Narrowing](#)
10. [tsconfig.json — Production Config](#)
11. [Modules & Namespaces](#)
12. [React + TypeScript Patterns](#)
13. [Advanced Patterns for Interviews](#)

14. 15 Interview Q&A

15. Where to Practice

1. Why TypeScript at 30 LPA

Benefit	Impact in interviews & production
Compile-time safety	Catch null/undefined, wrong props, API shape mismatches before deploy
IDE intelligence	Jump-to-definition, rename symbol, inline docs – faster refactors
Self-documenting APIs	Function signatures replace tribal knowledge
Scale on teams	50+ engineers touching same codebase without silent breakage
Ecosystem standard	React 18+, Next.js, NestJS, tRPC, Prisma all assume TS fluency

TypeScript is a **structural type system** — compatibility is based on shape, not explicit inheritance (duck typing with verification).

```
// JavaScript runtime – identical output after tsc
const greet = (name: string): string => `Hello, ${name}`;
```

2. Setup & Toolchain

2.1 Quick Start

```
# Global CLI (optional – prefer local devDependency)
npm install -D typescript @types/node

# Initialize config
npx tsc --init

# Compile once
npx tsc

# Watch mode
npx tsc --watch
```

2.2 Project Structure (MERN / React)

```
src/
  components/
  hooks/
  types/
    api.ts      # shared API response types
    user.ts
  utils/
  index.tsx
tsconfig.json
tsconfig.node.json # Vite/Next split configs
```

2.3 Tooling Stack

Tool	Purpose
<code>typescript</code>	Compiler (<code>tsc</code>)
<code>@types/react</code> , <code>@types/node</code>	Ambient type definitions for JS libraries
<code>tsx</code> / <code>ts-node</code>	Run TS without pre-build (scripts)
<code>eslint</code> + <code>@typescript-eslint</code>	Lint rules aligned with TS
<code>vite</code> / <code>next</code>	Bundler with built-in TS transpilation (no emit)

Interview Box: "Does TypeScript run in the browser?"

Answer: No. `tsc` (or esbuild/swc via bundler) strips types and emits JavaScript. Types exist only at compile time — zero runtime cost unless you use features that emit code (enums, decorators with certain configs).

3. All Core Types

3.1 Primitives

```
let username: string = "priya";
let age: number = 28;
let isVerified: boolean = true;
let nothing: null = null;
let notAssigned: undefined = undefined;
let id: bigint = 9007199254740991n;
let flag: symbol = Symbol("id");
```

3.2 Arrays, Tuples, Readonly

```
const nums: number[] = [1, 2, 3];
const tags: Array<string> = ["ts", "react"]; // equivalent syntax

// Tuple – fixed length + typed positions
type Point = [number, number];
const origin: Point = [0, 0];

// Readonly arrays
const frozen: readonly number[] = [1, 2, 3];
// frozen.push(4); // Error
```

3.3 Object Types

```
type User = {
  id: string;
  email: string;
  role?: "admin" | "user"; // optional
  readonly createdAt: Date; // cannot reassign after init
};

const u: User = {
  id: "1",
  email: "a@b.com",
  createdAt: new Date(),
};
```

3.4 any, unknown, never, void

Type	Meaning	When to use
any	Opt out of checking	Legacy migration only – avoid
unknown	Safe top type – must narrow	External JSON, user input
void	Function returns undefined	Event handlers, side effects
never	No possible value	Exhaustive checks, throw helpers

```
function parseJSON(raw: string): unknown {
  return JSON.parse(raw);
}

function assertNever(x: never): never {
  throw new Error(`Unexpected: ${x}`);
}

function log(msg: string): void {
  console.log(msg);
}
```

3.5 Union & Intersection

```
type ID = string | number;

type Employee = { name: string; dept: string };
type Contact = { email: string; phone: string };
type EmployeeContact = Employee & Contact;

// Discriminated union – pattern for API results
type ApiResponse<T> =
  | { status: "success"; data: T }
  | { status: "error"; message: string; code: number };

function handle<T>(res: ApiResponse<T>) {
  if (res.status === "success") {
    console.log(res.data); // narrowed
  } else {
    console.error(res.code, res.message);
  }
}
```

3.6 Index Signatures & Record

```
type StringMap = { [key: string]: string };

type Role = "admin" | "editor" | "viewer";
type Permissions = Record<Role, boolean>;

const perms: Permissions = {
  admin: true,
  editor: true,
  viewer: false,
};
```

4. Interfaces vs Types

4.1 Side-by-Side

```
interface IUser {
  id: number;
  name: string;
}

type TUser = {
  id: number;
  name: string;
};
```

4.2 Decision Matrix

Feature	interface	type
Object shapes	✓ Preferred	✓ Works
Extend / implements	✓ extends	✓ & intersection
Declaration merging	✓ Yes	✗ No
Unions / tuples	✗ No	✓ Yes
Mapped / conditional types	✗ No	✓ Yes
Primitives aliases	✗ No	✓ type ID = string

```
// Declaration merging – only interfaces
interface Window {
  myCustomProp: string;
}
interface Window {
  anotherProp: number;
}
// Window now has both

// Union – types only
type Result = Success | Failure;
```

4.3 Official Guidance (2024+)

- Default to **interface** for public object shapes (React props, API models).
- Use **type** for unions, tuples, utility compositions.

Interview Box: "Can you extend a type alias?"

Answer: You intersect with **&**, not **extends**. `type B = A & { extra: string }`. Interfaces use **extends** and merge when declared twice with the same name.

5. Functions & Overloads

```
// Basic
function add(a: number, b: number): number {
  return a + b;
}

// Optional & default params
function greet(name: string, greeting = "Hello"): string {
  return `${greeting}, ${name}`;
}

// Rest params
function sum(...nums: number[]): number {
  return nums.reduce((a, b) => a + b, 0);
}

// Function overloads – multiple call signatures, one implementation
function format(value: string): string;
function format(value: number): string;
function format(value: string | number): string {
  return String(value);
}

// Generic function
function identity<T>(arg: T): T {
  return arg;
}
```

Callback Typing

```
type ClickHandler = (event: React.MouseEvent<HTMLButtonElement>) => void;

type Fetcher<T> = (url: string) => Promise<T>;
```

6. Generics – Deep Dive

Generics parameterize types — write reusable, type-safe abstractions.

6.1 Generic Functions & Constraints

```
function first<T>(arr: T[]): T | undefined {
  return arr[0];
}

// Constraint - T must have length
function logLength<T extends { length: number }>(item: T): void {
  console.log(item.length);
}

// keyof constraint
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}
```

6.2 Generic Interfaces & Classes

```
interface Repository<T> {
  findById(id: string): Promise<T | null>;
  save(entity: T): Promise<T>;
}

class InMemoryRepo<T extends { id: string }> implements Repository<T> {
  private store = new Map<string, T>();

  async findById(id: string): Promise<T | null> {
    return this.store.get(id) ?? null;
  }

  async save(entity: T): Promise<T> {
    this.store.set(entity.id, entity);
    return entity;
  }
}
```

6.3 Generic Defaults

```
type ApiResult<T = unknown> = {  
  data: T;  
  timestamp: number;  
};  
  
type Paginated<T, PageSize extends number = 20> = {  
  items: T[];  
  pageSize: PageSize;  
  total: number;  
};
```

6.4 infer Keyword (Conditional Types)

```
type ReturnTypeOf<T> = T extends (...args: unknown[]) => infer R ? R : never;  
  
type FetchUser = () => Promise<{ id: string }>;  
type UserPromise = ReturnTypeOf<FetchUser>; // Promise<{ id: string }>
```

Interview Box: "Why not use `any` instead of generics?"

Answer: Generics preserve the relationship between input and output types. `identity<any>(x)` loses type info; `identity<string>("hi")` returns `string`. Critical for hooks like `useState<T>` and API clients.

7. Utility Types

Built-in helpers in `lib.es5.d.ts` — know these cold for interviews.

7.1 Common Utilities

```
interface User {
  id: string;
  name: string;
  email: string;
  password: string;
}

// Partial – all optional (PATCH payloads)
type UserUpdate = Partial<User>;

// Required – opposite
type RequiredUser = Required<User>;

// Pick & Omit
type PublicUser = Pick<User, "id" | "name">;
type SafeUser = Omit<User, "password">;

// Readonly
type FrozenUser = Readonly<User>;

// Record
type UserMap = Record<string, User>;
```

7.2 ReturnType, Parameters, Awaited

```
async function fetchUser(id: string) {
  return { id, name: "Amit" };
}

type FetchUserReturn = Awaited<ReturnType<typeof fetchUser>>;
// { id: string; name: string }

type FetchUserParams = Parameters<typeof fetchUser>; // [id: string]
```

7.3 Exclude, Extract, NonNullable

```
type T1 = Exclude<"a" | "b" | "c", "a">; // "b" | "c"  
type T2 = Extract<"a" | "b" | 1, string>; // "a" | "b"  
type T3 = NonNullable<string | null | undefined>; // string
```

7.4 Custom Mapped Types

```
type Optional<T> = {  
  [K in keyof T]?: T[K];  
};  
  
type Nullable<T> = {  
  [K in keyof T]: T[K] | null;  
};  
  
// Template literal keys  
type EventNames = "click" | "focus";  
type Handlers = {  
  [K in EventNames as `${Capitalize<K>}`]: () => void;  
};  
// { onClick: () => void; onFocus: () => void }
```

Utility Type Cheat Sheet

Utility	Transforms
<code>Partial<T></code>	All properties optional
<code>Required<T></code>	All properties required
<code>Readonly<T></code>	All properties readonly
<code>Pick<T, K></code>	Subset of keys
<code>Omit<T, K></code>	Remove keys
<code>Record<K, V></code>	Object type from keys → values
<code>Exclude<T, U></code>	Remove from union
<code>Extract<T, U></code>	Keep matching union members
<code>NonNullable<T></code>	Remove null/undefined
<code>ReturnType<F></code>	Function return type
<code>Parameters<F></code>	Function param tuple
<code>Awaited<T></code>	Unwrap Promise

8. Enums & Literal Types

8.1 String Literal Unions (Preferred in Modern TS)

```
type Status = "idle" | "loading" | "success" | "error";

function setStatus(s: Status) {
  /* ... */
}
```

8.2 const Object Pattern

```
const ROLES = {  
  ADMIN: "admin",  
  USER: "user",  
} as const;  
  
type Role = (typeof ROLES)[keyof typeof ROLES]; // "admin" | "user"
```

8.3 Enums (Know trade-offs)

```
enum Direction {  
  Up,  
  Down,  
  Left,  
  Right,  
}  
  
enum HttpStatus {  
  OK = 200,  
  NotFound = 404,  
  ServerError = 500,  
}  
  
const enum Compression {  
  None,  
  Gzip,  
} // Inlined at compile time – no runtime object
```

Approach	Runtime cost	Reverse mapping	Recommendation
String union	None	N/A	✅ Default choice
<code>as const</code> object	Minimal	N/A	✅ Config/constants
<code>enum</code>	Yes (object)	Numeric only	Use when team standard
<code>const enum</code>	Inlined	Numeric only	Bundler-dependent

9. Type Guards & Narrowing

9.1 typeof & instanceof

```
function padLeft(value: string | number): string {  
  if (typeof value === "number") {  
    return " ".repeat(value);  
  }  
  return value.padStart(10);  
}  
  
if (error instanceof Error) {  
  console.log(error.message);  
}
```

9.2 Truthiness & Equality Narrowing

```
function print(str: string | null | undefined) {  
  if (str) {  
    console.log(str.toUpperCase()); // string  
  }  
}
```

9.3 in Operator

```
type Fish = { swim: () => void };  
type Bird = { fly: () => void };  
  
function move(animal: Fish | Bird) {  
  if ("swim" in animal) {  
    animal.swim();  
  } else {  
    animal.fly();  
  }  
}
```

9.4 User-Defined Type Guards

```
interface Admin {
  role: "admin";
  permissions: string[];
}

interface Member {
  role: "member";
}

type Person = Admin | Member;

function isAdmin(p: Person): p is Admin {
  return p.role === "admin";
}

function authorize(p: Person) {
  if (isAdmin(p)) {
    return p.permissions.includes("write");
  }
  return false;
}
```

9.5 Discriminated Unions + Exhaustiveness

```
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "rect"; width: number; height: number };

function area(shape: Shape): number {
  switch (shape.kind) {
    case "circle":
      return Math.PI * shape.radius ** 2;
    case "rect":
      return shape.width * shape.height;
    default:
      return assertNever(shape);
  }
}
```

9.6 Assertion Functions

```
function assertIsString(val: unknown): asserts val is string {  
  if (typeof val !== "string") {  
    throw new Error("Not a string");  
  }  
}
```

10. tsconfig.json – Production Config

10.1 Strict MERN / React Baseline

```
{  
  "compilerOptions": {  
    "target": "ES2022",  
    "lib": ["ES2022", "DOM", "DOM.Iterable"],  
    "module": "ESNext",  
    "moduleResolution": "bundler",  
    "jsx": "react-jsx",  
    "strict": true,  
    "noUncheckedIndexedAccess": true,  
    "noImplicitOverride": true,  
    "exactOptionalPropertyTypes": true,  
    "skipLibCheck": true,  
    "esModuleInterop": true,  
    "isolatedModules": true,  
    "resolveJsonModule": true,  
    "declaration": true,  
    "declarationMap": true,  
    "sourceMap": true,  
    "outDir": "./dist",  
    "baseUrl": ".",  
    "paths": {  
      "@/*": ["src/*"]  
    }  
  },  
  "include": ["src"],  
  "exclude": ["node_modules", "dist"]  
}
```

10.2 Key Flags Explained

Flag	Effect
<code>strict</code>	Enables <code>strictNullChecks</code> , <code>noImplicitAny</code> , etc.
<code>noUncheckedIndexedAccess</code>	<code>arr[i]</code> is <code>T undefined</code>
<code>exactOptionalPropertyTypes</code>	<code>{ x?: string }</code> can't assign <code>undefined</code> explicitly
<code>moduleResolution: bundler</code>	Vite/Next – allows <code>import</code> without extension
<code>isolatedModules</code>	Each file transpilable alone (required for SWC/esbuild)

Interview Box: "What's the difference between `moduleResolution: node` and `bundler`?"

Answer: `node` follows Node10/Node16 resolution for published packages. `bundler` assumes a modern bundler handles resolution — supports `package.json` `"exports"`, extensionless imports, and aligns with Vite/Next/esbuild workflows.

11. Modules & Namespaces

11.1 ES Modules (Standard)

```
// user.ts
export interface User { id: string; name: string }
export const defaultRole = "member";
export default function createUser(name: string): User {
  return { id: crypto.randomUUID(), name };
}

// app.ts
import createUser, { User, defaultRole } from "./user";
import type { User as UserType } from "./user"; // type-only import
```

11.2 Type-Only Exports/Imports

```
export type { User };  
import type { ComponentProps } from "react";
```

Erased at compile — no runtime import emitted when using `import type`.

11.3 Ambient Declarations

```
// global.d.ts  
declare global {  
  interface Window {  
    analytics: { track: (event: string) => void };  
  }  
}  
export {};
```

11.4 Namespaces (Legacy – Avoid in New Code)

```
namespace Utils {  
  export function clamp(n: number, min: number, max: number) {  
    return Math.min(max, Math.max(min, n));  
  }  
}
```

Prefer ES modules. Namespaces appear in older codebases and DefinitelyTyped internals.

12. React + TypeScript Patterns

12.1 Component Props

```
import { type ReactNode, type ComponentPropsWithoutRef } from "react";

// Basic props
interface ButtonProps {
  label: string;
  variant?: "primary" | "secondary";
  disabled?: boolean;
  onClick?: () => void;
}

function Button({ label, variant = "primary", disabled, onClick }: ButtonProps) {
  return (
    <button className={variant} disabled={disabled} onClick={onClick}>
      {label}
    </button>
  );
}

// Children
interface CardProps {
  title: string;
  children: ReactNode;
}

// Extending native elements
type InputProps = ComponentPropsWithoutRef<"input"> & {
  label: string;
  error?: string;
};

function Input({ label, error, ...rest }: InputProps) {
  return (
    <label>
      {label}
      <input {...rest} />
      {error && <span className="error">{error}</span>}
    </label>
  );
}
```

12.2 Hooks

```
import { useState, useEffect, useReducer, useRef, useCallback } from "react";

// useState with explicit type
const [user, setUser] = useState<User | null>(null);

// useReducer
type State = { count: number };
type Action = { type: "inc" } | { type: "dec" } | { type: "reset" };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "inc": return { count: state.count + 1 };
    case "dec": return { count: state.count - 1 };
    case "reset": return { count: 0 };
  }
}

// Custom hook – return type inferred or explicit
function useLocalStorage<T>(key: string, initial: T) {
  const [value, setValue] = useState<T>(() => {
    const stored = localStorage.getItem(key);
    return stored ? (JSON.parse(stored) as T) : initial;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue] as const;
}
```

12.3 Event Handlers

```
function SearchForm() {
  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    console.log(e.target.value);
  };

  const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
  };

  return (
    <form onSubmit={handleSubmit}>
      <input onChange={handleChange} />
    </form>
  );
}
```

12.4 Generic Components

```
interface ListProps<T> {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
  keyExtractor: (item: T) => string;
}

function List<T>({ items, renderItem, keyExtractor }: ListProps<T>) {
  return (
    <ul>
      {items.map((item) => (
        <li key={keyExtractor(item)}>{renderItem(item)}</li>
      ))}
    </ul>
  );
}
```

12.5 Context

```
interface AuthContextValue {
  user: User | null;
  login: (email: string, password: string) => Promise<void>;
  logout: () => void;
}

const AuthContext = createContext<AuthContextValue | undefined>(undefined);

function useAuth(): AuthContextValue {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used within AuthProvider");
  return ctx;
}
```

13. Advanced Patterns for Interviews

13.1 Branded Types

```
type UserId = string & { readonly __brand: unique symbol };
type Email = string & { readonly __brand: unique symbol };

function createUserId(id: string): UserId {
  return id as UserId;
}

function getUser(id: UserId) { /* ... */ }

// getUser("raw-string"); // Error if not branded
```

13.2 satisfies Operator

```
type Colors = Record<string, `#${string}`>;

const palette = {
  primary: "#ff0000",
  secondary: "#00ff00",
} satisfies Colors;

// palette.primary is "#ff0000" (literal), not just string
```

13.3 Template Literal Types

```
type HTTPMethod = "GET" | "POST" | "PUT" | "DELETE";
type Endpoint = `/api/${string}`;
type Route = `${HTTPMethod} ${Endpoint}`;
```

13.4 Module Augmentation

```
// types/express.d.ts
import "express";

declare module "express-serve-static-core" {
  interface Request {
    user?: { id: string; role: string };
  }
}
```

14. 15 Interview Q&A

Q1: What is structural typing?

TypeScript checks shape compatibility, not nominal declarations. Two types with the same structure are assignable even if declared separately.

Q2: `interface` vs `type` — when do you pick each?

Interface for extendable object contracts (props, entities). Type for unions, mapped types, tuples, and conditional logic.

Q3: Difference between `any` and `unknown` ?

`any` disables checking; you can call anything on it. `unknown` forces narrowing before use — safer for external data.

Q4: What does `strictNullChecks` do?

`null` and `undefined` are not assignable to other types unless explicitly included (`string | null`). Eliminates a huge class of runtime bugs.

Q5: Explain generics with a real example.

`Array<T>` — push and pop preserve element type. `fetch<User>('/api/user')` returns `Promise<User>` without casting.

Q6: What is a discriminated union?

Union members share a literal discriminator field (`kind` , `status`). Switch/if narrows to exact member type.

Q7: How does `Pick` differ from `Omit` ?

`Pick<T, K>` keeps listed keys. `Omit<T, K>` removes listed keys. `Omit<User, 'id'> = Pick<User, Exclude<keyof User, 'id'>>` .

Q8: Why avoid numeric enums in new code?

They emit runtime objects, allow reverse mapping (unexpected values), and don't tree-shake as cleanly as string unions + `as const` .

Q9: What is `noUncheckedIndexedAccess` ?

Index access returns `T | undefined` because JS arrays/objects may lack that key. Forces defensive checks.

Q10: Explain `ReturnType` and `Parameters` .

Extract return type or parameter tuple from a function type/value. Used to derive types from existing functions without duplication.

Q11: What is declaration merging?

Multiple `interface` declarations with the same name merge. Used to extend third-party types (e.g., Express `Request`).

Q12: `import type` vs regular import?

`import type` guarantees erasure — no runtime import. Prevents circular dependency issues and accidental value imports.

Q13: How do assertion functions work?

`asserts val is string` tells TS the param is narrowed after call if function returns normally (throws otherwise).

Q14: What is the `satisfies` operator?

Validates a value matches a type while preserving literal inference. Better than `: Type` annotation when you want narrow literals.

Q15: How do you type a React `forwardRef` component?

`const Input = forwardRef<HTMLInputElement, InputProps>((props, ref) => ...)` . Ref type is first generic, props second.

15. Where to Practice

Official & Structured Learning

Resource	URL	Focus
TypeScript Handbook	typescriptlang.org/docs/handbook	Canonical reference – read cover to cover
TS Playground	typescriptlang.org/play	Experiment with compiler options live
Total TypeScript	totaltypescript.com	Matt Pocock – generics, transforms, puzzles
Type Challenges	github.com/type-challenges/type-challenges	LeetCode for types – interview gold

LeetCode / Interview Style

Platform	Approach
LeetCode	Solve JS problems, then strict rewrites in TS with full types
Exercism TypeScript track	Small exercises with mentor feedback
Total TypeScript Turbo	Paid – daily type puzzles mirroring senior interviews

Weekly Practice Plan (30 LPA Track)

Week	Activity
1-2	Handbook + Playground – all primitive types, narrowing
3-4	Generics + utility types – implement <code>MyPick</code> , <code>MyPartial</code>
5	React + TS – type 3 components with <code>forwardRef</code> , generic <code>List</code>
6	Type Challenges (Easy → Medium) – 20 problems
7+	Refactor a MERN feature to strict TS with <code>noUncheckedIndexedAccess</code>

Mock Interview Prompts

1. Design types for a paginated REST API with error codes.
2. Implement `deepPartial<T>` without built-in utilities.
3. Type a Redux slice or Zustand store from scratch.
4. Explain how you'd migrate a 50k LOC JS codebase to strict TS.

Next in bootcamp: [03-react-complete.md](#) — React 18 with TypeScript integration.

Product Company Interview Bank – TypeScript (50 Questions)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Target companies: Google, Meta, Microsoft, Amazon, Adobe, Atlassian, Uber, Flipkart, Salesforce, Stripe **Rule:** Full terms only — no unexplained abbreviations. Write for absolute beginners who know nothing.

Question 1: What is TypeScript and why do product companies use it instead of plain JavaScript?

Answer: TypeScript is a programming language created by Microsoft that adds optional static type checking on top of JavaScript. You write TypeScript code, and a compiler turns it into regular JavaScript that browsers and Node.js can run. Product companies use it because types catch many bugs before code runs, improve editor autocomplete, and make large codebases easier to refactor. At companies like Google and Stripe, type safety reduces production incidents and speeds up code review.

Question 2: What is the output of this code, and why?

```
let value: string | number = "hello";
console.log(typeof value);
```

Answer: The output is `"string"`. Even though `value` is declared with a union type (`string | number`), at runtime JavaScript only sees the actual value assigned, which is the string `"hello"`. TypeScript types exist only at compile time and are erased when the code is compiled to JavaScript. The `typeof` operator checks the runtime value, not the TypeScript type annotation.

Question 3: What is the difference between `interface` and `type` in TypeScript?

Answer: Both describe the shape of an object, but they behave differently in advanced cases. Interfaces can be extended with `extends` and merged when declared twice with the same name (declaration merging). Type aliases can represent unions, tuples, and mapped types that interfaces cannot express as cleanly. For plain object shapes, either works; many teams prefer `interface` for public application programming interfaces and `type` for unions and computed types.

Question 4: What does the `strict` option in `tsconfig.json` do?

Answer: Setting `"strict": true` enables a bundle of strict type-checking rules, including `strictNullChecks`, `noImplicitAny`, and others. With strict mode on, you must handle `null` and `undefined` explicitly, and TypeScript will error when it cannot infer a type. Product companies almost always enable strict mode because it prevents an entire class of null-reference and implicit-any bugs.

Question 5: Predict the compile error (if any) for this code:

```
function greet(name: string) {  
  return "Hello, " + name;  
}  
greet(42);
```

Answer: TypeScript reports a compile error: `Argument of type 'number' is not assignable to parameter of type 'string'`. The function expects a string, but `42` is a number. This error appears at compile time, so the bug never reaches production. Fixing it requires passing a string like `greet("Alice")` or changing the parameter type if numbers are valid input.

Question 6: What is type narrowing, and how does `typeof` help with it?

Answer: Type narrowing is the process of refining a broad type to a more specific type inside a block of code. When a variable has a union type like `string | number`, you can use `typeof value === "string"` inside an `if` block, and TypeScript knows `value` is a string inside that block. This lets you call string-only methods safely without extra casts.

Question 7: Implement a generic function `identity` that returns whatever you pass in, preserving the input type.

Answer:

```
function identity<T>(value: T): T {  
  return value;  
}  
  
const num = identity(42);           // inferred as number  
const str = identity("hello");      // inferred as string
```

The generic type parameter `T` stands in for the actual type at each call site. TypeScript infers `T` from the argument, so the return type always matches the input type without manual annotation.

Question 8: What is `keyof`, and give a practical example.

Answer: `keyof` produces a union of all property names on a type. For example, `keyof { id: number; name: string }` becomes `"id" | "name"`. A common use is building type-safe property accessors:

```
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {  
  return obj[key];  
}
```

This ensures you can only pass valid keys and get the correct return type for each key.

Question 9: What does `Partial<T>` do?

Answer: `Partial<T>` is a built-in utility type that makes every property of `T` optional. If you have `interface User { id: number; name: string }`, then `Partial<User>` becomes `{ id?: number; name?: string }`. It is commonly used for update functions where you may change only some fields: `function updateUser(id: number, changes: Partial<User>) { ... }`.

Question 10: What is the difference between `enum` and a string union type like `"pending" | "done"` ?

Answer: An `enum` creates a real JavaScript object at runtime with named members and numeric or string values. A string union type exists only at compile time and produces no extra runtime code. Many modern codebases prefer string unions because they are simpler, tree-shake better, and avoid enum quirks like reverse mapping on numeric enums. Use enums when you need runtime iteration over all values.

Question 11: What will this code log?

```
const config = {  
  host: "localhost",  
  port: 3000,  
} as const;  
  
console.log(config.port);  
config.port = 4000;
```

Answer: It logs `3000`, but the second line causes a compile error: `Cannot assign to 'port' because it is a read-only property`. The `as const` assertion makes every property deeply readonly and narrows literal types (`"localhost"` instead of `string`, `3000` instead of `number`). This pattern is popular for constant configuration objects.

Question 12: Explain `satisfies` and how it differs from a type annotation.

Answer: The `satisfies` operator checks that a value matches a type while preserving the value's inferred literal types. With a type annotation like `const colors: Record<string, string> = { red: "#f00" }`, you lose the specific keys. With `const colors = { red: "#f00" } satisfies Record<string, string>`, TypeScript verifies the shape but still knows `colors.red` exists. This gives both validation and precise autocomplete.

Question 13: What is structural typing (duck typing) in TypeScript?

Answer: TypeScript uses structural typing: two types are compatible if their members match, regardless of their names. If `interface Point { x: number; y: number; }` and an object `{ x: 1, y: 2, z: 3 }` is passed where `Point` is expected, extra properties on the object are often

allowed when passed directly. This differs from nominal typing in languages like Java, where type names must match exactly.

Question 14: Debug this code – why does TypeScript complain?

```
interface ApiResponse {  
  data: unknown;  
}  
  
const response: ApiResponse = { data: { id: 1, name: "Ada" } };  
console.log(response.data.name);
```

Answer: The error is `Object is of type 'unknown'`. Property `data` is typed as `unknown`, so you cannot access `.name` without first narrowing or asserting the type. Fix by narrowing:

```
if (typeof response.data === "object" && response.data !== null && "name" in response.data) {  
  console.log((response.data as { name: string }).name);  
}
```

Or define a proper generic: `interface ApiResponse<T> { data: T }.`

Question 15: What is `Pick<T, K>` and when would you use it?

Answer: `Pick<T, K>` creates a new type with only the properties listed in `K`. Example: `Pick<User, "id" | "name">` gives `{ id: number; name: string }`. Use it when an function should accept or return a subset of a larger interface, such as a list view that needs only id and name from a full User object.

Question 16: What does `"moduleResolution": "bundler"` mean in modern TypeScript projects?

Answer: It tells TypeScript how to resolve import paths to files. The `"bundler"` setting matches how tools like Vite, Webpack, and esbuild resolve modules, including support for extensionless imports and package.json `"exports"` fields. Product teams using Vite or Next.js often set this alongside `"module": "ESNext"` for accurate import checking.

Question 17: Implement a type-safe event emitter with typed event names and payloads.

Answer:

```
type EventMap = {
  click: { x: number; y: number };
  load: { durationMs: number };
};

class TypedEmitter<T extends Record<string, unknown>> {
  private listeners: Partial<{ [K in keyof T]: Array<(payload: T[K]) => void> }> = {}

  on<K extends keyof T>(event: K, handler: (payload: T[K]) => void) {
    (this.listeners[event] ??= []).push(handler);
  }

  emit<K extends keyof T>(event: K, payload: T[K]) {
    this.listeners[event]?.forEach((handler) => handler(payload));
  }
}
```

This pattern appears in interview questions about generics and mapped types at companies like Atlassian and Uber.

Question 18: What is the `never` type used for?

Answer: `never` represents values that should never occur. Functions that always throw or run forever return `never`. In control flow, after exhaustive checks, a variable may be narrowed to `never`. It is also used in conditional types to filter types out. Example: a switch on a union should have a `default` case assigning to `never` to catch missing cases at compile time.

Question 19: What is type inference, and where does TypeScript infer types automatically?

Answer: Type inference means the compiler deduces types without explicit annotations. It infers variable types from initializers (`const x = 5` → `number`), function return types from return statements, and generic type parameters from arguments. Good inference reduces boilerplate while keeping safety. You only annotate when inference is too wide or ambiguous.

Question 20: What is the output type of `ReturnType<typeof fetchUser>` if `fetchUser` is declared as:

```
async function fetchUser(id: string) {  
  return { id, name: "Bob" };  
}
```

Answer: `ReturnType<typeof fetchUser>` is `Promise<{ id: string; name: string }>`. `typeof fetchUser` gets the function type, and `ReturnType` extracts its return type. Because the function is `async`, the return type is wrapped in `Promise`. This utility is useful when you want to reuse API response shapes without duplicating interface definitions.

Question 21: What are decorators in TypeScript (basics)?

Answer: Decorators are experimental syntax (prefix `@`) that attach metadata or modify classes, methods, properties, or parameters. They are enabled with `"experimentalDecorators": true` in `tsconfig.json`. Frameworks like Angular and some Node.js libraries use them for dependency injection and routing. Decorators run at class definition time and are a Stage 3 ECMAScript proposal; understand the concept even if your team prefers higher-order functions instead.

Question 22: Fix the type error without using `any`:

```
function merge(a: object, b: object) {  
  return { ...a, ...b };  
}  
const result = merge({ x: 1 }, { y: "two" });  
console.log(result.x);
```

Answer: The return type of `merge` is inferred as `{}`, so `result.x` errors. Fix with generics:

```
function merge<A extends object, B extends object>(a: A, b: B): A & B {  
  return { ...a, ...b };  
}
```

Now `result` has type `{ x: number } & { y: string }`, and `result.x` is valid.

Question 23: What is `Omit<T, K>` ?

Answer: `Omit<T, K>` removes specified keys from type `T`. Example: `Omit<User, "password">` creates a public user type without the password field. This is common in application programming interfaces that return user data to the client while keeping sensitive fields out of the type.

Question 24: Explain `strictNullChecks` with an example.

Answer: With `strictNullChecks` enabled, `null` and `undefined` are not assignable to other types unless explicitly allowed. Without it, `let name: string = null` might be allowed. With it, you must write `let name: string | null = null` and check before use:

```
function printLength(text: string | null) {  
  if (text === null) return;  
  console.log(text.length); // safe: text is string here  
}
```

This prevents the most common production crash: reading a property of `null`.

Question 25: What is a mapped type? Give an example.

Answer: A mapped type transforms each property of an existing type. Example — make all properties readonly:

```
type Readonly<T> = {  
  readonly [K in keyof T]: T[K];  
};
```

Built-in utilities like `Partial`, `Pick`, and `Record` are mapped types. Interviewers at Microsoft and Meta often ask you to implement a simplified version by hand.

Question 26: How do you type React component props in TypeScript?

Answer: Define an interface or type for props and use it on the function component:

```
interface ButtonProps {  
  label: string;  
  onClick: () => void;  
  disabled?: boolean;  
}  
  
function Button({ label, onClick, disabled = false }: ButtonProps) {  
  return <button disabled={disabled} onClick={onClick}>{label}</button>;  
}
```

For children, add `children?: React.ReactNode`. Product companies expect you to avoid `any` on props and event handlers.

Question 27: What does `Record<K, V>` do?

Answer: `Record<K, V>` builds an object type whose keys are `K` and values are `V`. Example: `Record<"USD" | "EUR", number>` means an object with currency keys and numeric values. It is cleaner than `{ [key: string]: number }` when keys are a known union, such as theme color names or supported locales.

Question 28: Predict the error:

```
type Status = "idle" | "loading" | "success";  
const s: Status = "pending";
```

Answer: Compile error: Type `"pending"` is not assignable to type `Status`. The string `"pending"` is not in the union. Union types restrict values to an explicit list, catching typos and invalid states at compile time — a core pattern for state machines in front-end applications.

Question 29: What is declaration merging?

Answer: Declaration merging lets TypeScript combine multiple declarations with the same name. Interfaces merge automatically:

```
interface User { id: number }  
interface User { name: string }  
// User is now { id: number; name: string }
```

This is how you extend third-party types in `.d.ts` files. Type aliases do not merge; duplicate type names are errors.

Question 30: How does TypeScript integrate with Node.js projects?

Answer: Install `@types/node` for Node.js global types (`process`, `Buffer`, file system modules). Set `"module"` and `"moduleResolution"` appropriately (often `"NodeNext"` for modern Node). Use `"types": ["node"]` in `tsconfig.json` if needed. Run with `ts-node` for development or compile to JavaScript with `tsc` for production. Stripe and Amazon backend teams expect familiarity with typing Express or Fastify handlers.

Question 31: What is a conditional type? Explain `T extends U ? X : Y`.

Answer: A conditional type picks one of two types based on a condition:

```
type IsString<T> = T extends string ? true : false;  
type A = IsString<"hi">; // true  
type B = IsString<number>; // false
```

Conditional types power advanced utilities like `Exclude` and `Extract`. Interviewers may ask you to read (not necessarily write) nested conditional types on the whiteboard.

Question 32: What is the difference between `interface extends` and intersection types (`&`)?

Answer: Both combine types, but intersections work on any types (including unions and primitives wrapped in objects), while `extends` is interface-specific syntax. `interface Admin extends User { role: string }` is readable for object inheritance. `type Admin = User & { role: string }` achieves a similar result with type aliases. Intersections can produce impossible types (e.g., `string & number` → `never`) that error immediately.

Question 33: Implement `Exclude<T, U>` using conditional types.

Answer:

```
type MyExclude<T, U> = T extends U ? never : T;
```

If `T` is assignable to `U`, the result is `never` (filtered out); otherwise `T` remains. For a union input, distributive conditional types apply the check to each member. Example: `MyExclude<"a" | "b" | "c", "a"> → "b" | "c"`.

Question 34: What is `noImplicitAny` and why does it matter?

Answer: When enabled, TypeScript errors if it would infer `any` for a variable, parameter, or return type without an explicit annotation. This stops silent loss of type safety. For example, a function parameter with no type and no inference context becomes an error instead of implicitly `any`. Strict teams treat explicit `any` as a code-review red flag.

Question 35: What will this print, and is there a type issue?

```
const tuple: [string, number] = ["age", 30];  
const [key, val] = tuple;  
console.log(key, val);
```

Answer: It prints `age 30`. Types are correct: destructuring preserves `string` and `number`. A common mistake is assigning `tuple.push(true)` — that errors because the tuple allows exactly two elements of fixed types, unlike a regular array.

Question 36: How do you type a `JSON.parse` result safely?

Answer: `JSON.parse` returns `any` by default. Never trust it blindly. Define an expected interface and validate at runtime with a schema library (Zod, io-ts) or manual guards:

```
interface User { id: number; name: string }

function parseUser(json: string): User {
  const raw: unknown = JSON.parse(json);
  if (typeof raw !== "object" || raw === null) throw new Error("Invalid");
  const obj = raw as Record<string, unknown>;
  if (typeof obj.id !== "number" || typeof obj.name !== "string") throw new Error(
  return { id: obj.id, name: obj.name };
}
```

Salesforce and Adobe interviews emphasize runtime validation, not just compile-time types.

Question 37: What is the `unknown` type versus `any`?

Answer: Both hold any value, but `any` disables type checking — you can do anything with it. `unknown` forces you to narrow or assert before use, making it the type-safe escape hatch. Prefer `unknown` for external data (API responses, user input). Use `any` only as a last resort with a comment explaining why.

Question 38: Explain variance briefly: why is this assignment invalid?

```
let read: Array<string> = ["a"];
let write: Array<string | number> = read; // error in strict mode
```

Answer: TypeScript arrays are mutable, so they are invariant in their element type. If this assignment were allowed, you could push a number into `write`, breaking `read`'s string-only guarantee. Readonly arrays can be covariant in some cases. Understanding variance comes up in advanced generic library design interviews.

Question 39: What does `"isolatedModules": true` enforce?

Answer: Each file must be translatable to JavaScript independently, which tools like Babel and esbuild require. It disallows features that need whole-program analysis, such as `const enum` inlining expectations without proper flags, and re-exporting types without `export type`. Create React App, Vite, and Next.js projects typically enable this.

Question 40: Implement a discriminated union for a payment result.

Answer:

```
type PaymentResult =
  | { status: "success"; transactionId: string }
  | { status: "failure"; errorCode: number; message: string };

function handlePayment(result: PaymentResult) {
  switch (result.status) {
    case "success":
      console.log(result.transactionId); // TypeScript knows this field exists
      break;
    case "failure":
      console.log(result.errorCode);
      break;
  }
}
```

The shared literal field `status` acts as a discriminant so TypeScript narrows the union in each branch. This pattern is standard at Flipkart and Uber for modeling API responses.

Question 41: What is `Required<T>` ?

Answer: `Required<T>` makes every property of `T` mandatory by removing optional modifiers. If `interface Config { host?: string; port?: number }`, then `Required<Config>` requires both `host` and `port`. Useful when you have validated that defaults were applied and want to pass a complete object downstream.

Question 42: How do path aliases work in TypeScript (`"@/components/Button"`)?

Answer: In `tsconfig.json`, set `"baseUrl"` and `"paths"`:

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": { "@/*": ["src/*"] }
  }
}
```

TypeScript uses this for type resolution and editor navigation. Your bundler (Vite, Webpack) must also resolve the same aliases at build time. Misaligned alias config is a common source of "module not found" bugs.

Question 43: What is the difference between `.ts`, `.tsx`, `.d.ts`, and `.mts` files?

Answer: `.ts` is standard TypeScript. `.tsx` allows JSX syntax for React components. `.d.ts` declares types only — no implementation — for JavaScript libraries or ambient types. `.mts` is ECMAScript module TypeScript used in Node.js `"type": "module"` packages. Knowing file extensions matters when configuring `include` and `exclude` in `tsconfig.json`.

Question 44: Debug: why might this React hook fail type checking?

```
function useCounter(initial: number) {  
  const [count, setCount] = useState(initial);  
  return { count, increment: () => setCount(count + 1) };  
}
```

Answer: This often passes type checking, but the bug is stale closure: `increment` captures old `count`. The type issue interviewers add is sometimes `useState<number>()` without an initializer when TypeScript cannot infer. Fix stale state with functional updates: `setCount((c) => c + 1)`. Explicit generic: `useState<number>(initial)` when `initial` might be undefined.

Question 45: What is `NonNullable<T>` ?

Answer: `NonNullable<T>` removes `null` and `undefined` from `T`. Example: `NonNullable<string | null | undefined>` → `string`. Handy after a filter or guard when you know a value is present but TypeScript has not narrowed yet: `users.filter((u): u is NonNullable<typeof u> => u !== null)`.

Question 46: Company-style follow-up: How would you migrate a 500-file JavaScript codebase to TypeScript incrementally?

Answer: Enable `"allowJs": true` and `"checkJs": false` initially so JavaScript files compile alongside TypeScript. Rename files module by module from `.js` to `.ts`, starting with leaf

utilities with no dependents. Add `"strict": true` only when ready, or use `"strict"` per directory via project references. Add types for third-party packages via `@types/*`. Track progress with `"noImplicitAny"` on new files. Meta and Google teams value pragmatic incremental migration over big-bang rewrites.

Question 47: What does `as` (type assertion) do, and when should you avoid it?

Answer: `as` tells the compiler to treat a value as a specific type: `const el = document.getElementById("root") as HTMLDivElement`. It does not perform runtime conversion or validation. Avoid assertions when runtime data might not match — use narrowing instead. Double assertions (`as unknown as T`) are a smell that usually means the type model needs fixing.

Question 48: Predict the inferred type:

```
const actions = {  
  increment: (n: number) => n + 1,  
  decrement: (n: number) => n - 1,  
} as const;
```

Answer: Without `as const`, methods are typed normally. With `as const` on the object, the object becomes deeply readonly and method names are literal keys. The functions themselves remain `(n: number) => number`. Interviewers test whether you confuse `as const` on objects with `as const` on individual string literals.

Question 49: How do you share types between a React front end and a Node.js back end in a monorepo?

Answer: Put shared interfaces in a package (e.g., `packages/shared-types`) exported as TypeScript source or compiled declarations. Reference it in both apps via workspace protocol (`"@myapp/shared": "workspace:*"`). Use `export type` for type-only exports to avoid bundling issues. Tools like Turborepo and Nx use this pattern at Stripe and Salesforce scale.

Question 50: Company-style follow-up: What TypeScript question would you

ask a senior candidate, and what makes a strong answer?

Answer: Ask them to design a type-safe query builder or API client where method chains enforce valid column names and filter operators. A strong answer uses generics, `keyof`, conditional types, and explains tradeoffs (compile time vs complexity vs developer experience). They should mention testing types with `@ts-expect-error`, documenting public types, and avoiding leaked `any` from third-party libraries. Weak answers rely only on basic interfaces without discussing inference, strictness, or migration strategy.